SMARTRECOMENDA

RELATÓRIO ACADÊMICO DO PROJETO

Sistema demonstrativo de recomendação personalizada e explicável

Documento técnico-acadêmico

Instituição: Universidade Iguaçu (UNIG)

Curso / disciplina: Ciência da Computação – Tecnologias Emergentes e Aplicações

Autor(a): Salatiel Muzi Martins

Orientador(a): Denise Moraes do Nascimento Vieira

2026

Resumo

Este relatório apresenta o SmartRecomenda, uma aplicação web acadêmica para descoberta de filmes, séries, músicas e notícias demonstrativas. O problema abordado é a dificuldade de explorar catálogos heterogêneos sem uma curadoria ajustável e compreensível. A solução reúne uma interface responsiva em React e TypeScript, uma API em FastAPI, persistência local em SQLite e um algoritmo híbrido determinístico que combina preferências explícitas, sinais de uso, avaliações agregadas e popularidade. O usuário pode ajustar temas, tipos e idiomas, salvar favoritos, avaliar itens, consultar explicações e ocultar conteúdos; o acesso de avaliação acadêmica pode ocorrer sem login, em um perfil compartilhado sem privilégios administrativos. A metodologia organiza descoberta, especificação, prototipação, implementação e avaliação técnica. Na verificação executada em 8 de outubro de 2026, os 10 testes automatizados do backend passaram, a checagem de tipos TypeScript concluiu sem erros e o build de produção do Vite foi concluído. Esses resultados comprovam funcionamento técnico básico, não eficácia de recomendação em produção nem usabilidade validada com participantes. Persistem limitações importantes: catálogo fictício pequeno, sinais compartilhados na demonstração, ausência de estudo com usuários e métricas offline de relevância ainda não calculadas.

Palavras-chave: sistemas de recomendação; personalização; transparência; React; FastAPI; SQLite.

Sumário

1. Introdução

2. Objetivos

3. Fundamentação teórica

4. Metodologia

5. Persona e levantamento de necessidades

6. Especificação de requisitos

7. Prototipação

8. Arquitetura e tecnologias

9. Padrões de projeto

10. Análise de riscos

11. Viabilidade técnica e econômica

12. Trade-offs e critérios de avaliação

13. Testes e resultados

14. Conclusão

15. Referências

Nota: o sumário é textual para permanecer estável em leitores diferentes.

1. Introdução

Serviços digitais oferecem catálogos extensos e diversos, mas volume não equivale a facilidade de escolha. Filtros convencionais ajudam a localizar atributos conhecidos; ainda assim, podem exigir que a pessoa já saiba o que procura e pouco explicam por que um item foi destacado. Sistemas de recomendação procuram reduzir esse esforço ao ordenar alternativas segundo preferências e evidências de interação, sem substituir a decisão do usuário [1].

O SmartRecomenda responde a esse problema como protótipo acadêmico: apresenta um catálogo demonstrativo multicategoria e uma curadoria que expõe motivos legíveis. A questão de projeto é: como oferecer uma experiência simples de descoberta personalizada, com sinais controláveis e explicações, usando uma implementação leve que possa ser executada e avaliada em ambiente acadêmico?

A justificativa combina valor didático e valor de produto. O domínio permite demonstrar fluxo web completo — interface, API, persistência, autenticação opcional, autorização, interação e medição — sem depender de dados comerciais. A opção de visitante acadêmico reduz barreiras para avaliação; em contrapartida, seu perfil é compartilhado e não representa isolamento individual. Por isso, o sistema é descrito como demonstração e não como serviço de recomendação comercial.

2. Objetivos

2.1 Objetivo geral

Projetar, implementar e verificar tecnicamente uma aplicação web demonstrativa que personalize e explique recomendações de conteúdos, permitindo ao usuário ajustar seus sinais e explorar o catálogo com autonomia.

2.2 Objetivos específicos

- Oferecer catálogo demonstrativo pesquisável, com filtros por tipo, gênero e idioma.

- Persistir preferências, avaliações, favoritos e interações de navegação em banco relacional leve.
- Combinar correspondência explícita, sinais históricos, avaliação agregada e popularidade em um ranking híbrido.
- Apresentar explicações textuais para tornar observável parte da lógica de ordenação.
- Permitir ocultar itens e limpar sinais de navegação, mantendo avaliações e favoritos separados.
- Disponibilizar acesso de visitante acadêmico sem login, sem conceder privilégios administrativos.
- Proteger endpoints administrativos e o fluxo de autenticação OAuth opcional.
- Verificar contratos e fluxos principais por testes automatizados, checagem estática e build de produção.
- Identificar limites do protótipo e propor avaliação futura com usuários e métricas de ranking.

3. Fundamentação teórica

3.1 Sistemas de recomendação

Um sistema de recomendação estima quais itens podem ser relevantes para um perfil e os ordena para apoiar a descoberta. Estratégias baseadas em conteúdo usam atributos dos itens; estratégias colaborativas exploram padrões entre usuários; abordagens híbridas combinam fontes de evidência e podem reduzir limitações de uma única técnica [1]. O SmartRecomenda usa atributos (tipo, gênero, idioma), preferências declaradas, sinais do próprio perfil e indicadores agregados do catálogo. Não implementa filtragem colaborativa entre usuários nem treinamento de modelo de aprendizado de máquina.

A escolha híbrida é coerente com o catálogo pequeno e com a necessidade de explicabilidade, mas os pesos são regras heurísticas, não parâmetros aprendidos ou validados estatisticamente. O estudo clássico de Burke discute formas de combinar

recomendações híbridas e seus compromissos [2]. A literatura também alerta para cold-start, popularidade desigual, cobertura e diversidade; esses fatores orientam a análise de limitações e a avaliação futura.

3.2 Design Thinking

Design Thinking estrutura a investigação do problema a partir das pessoas, geração de alternativas, prototipação e teste iterativo. Brown enfatiza a integração entre desejabilidade, viabilidade técnica e viabilidade econômica [3]. Neste projeto, o processo é aplicado como roteiro de desenvolvimento e não como evidência de pesquisa de campo: a persona é uma proto-persona baseada no cenário de avaliação acadêmica, ainda sujeita a validação com participantes.

3.3 Engenharia de requisitos

A engenharia de requisitos transforma necessidades em requisitos verificáveis, explicita restrições e mantém rastreabilidade entre problema, implementação e teste. A ISO/IEC/IEEE 29148:2018 oferece orientação para processos e artefatos de requisitos [4]. Neste relatório, requisitos funcionais e não funcionais recebem identificadores e critérios observáveis; os critérios ainda não cobertos pelos testes são tratados como trabalho futuro, não como resultados.

3.4 Qualidade, usabilidade e padrões

A avaliação de qualidade deve considerar características como adequação funcional, confiabilidade, usabilidade, eficiência, segurança e manutenibilidade, conforme o modelo ISO/IEC 25010:2023 [5]. As heurísticas de Nielsen — visibilidade do estado, correspondência com o mundo real, controle, consistência, prevenção e recuperação de erros, entre outras — fornecem uma lista útil para inspeção preliminar da interface [6]. Elas não substituem teste de usabilidade com pessoas.

Padrões de projeto oferecem vocabulário para responsabilidades e colaboração entre componentes [7]. SOLID descreve princípios para modularidade e evolução [8]. O relatório avalia o grau de aplicação observado no código, evitando declarar conformidade integral: a API atual concentra boa parte das responsabilidades em

um módulo Python, e os rótulos Strategy e Observer representam simplificações concretas do MVP.

4. Metodologia

4.1 Processo de desenvolvimento

Adotou-se um processo incremental orientado por etapas: (1) delimitação do problema e dos perfis; (2) levantamento de necessidades e escopo; (3) desenho do fluxo e estrutura visual; (4) modelagem de dados e contratos da API; (5) implementação de interface, regras e persistência; (6) verificação automatizada e correções; (7) documentação de riscos, resultados e evolução. O processo aproxima-se de uma aplicação didática de Design Thinking e engenharia de requisitos, mas não houve pesquisa etnográfica, experimento controlado ou comparação estatística.

4.2 Recursos e ambiente

Recurso	Uso no projeto
React 19 + TypeScript 7	Interface de página única, componentes e contratos tipados.
Vite 8	Servidor local de desenvolvimento e compilação do frontend.
Python 3.12 + FastAPI 0.115	API REST, validação de payloads e documentação OpenAPI.
SQLite + sqlite3	Persistência local e consultas parametrizadas do protótipo.
Pytest + TestClient	Testes automatizados de endpoints e regras principais.
Vercel + Railway (configuração)	Caminho documentado para hospedagem separada; deploy não confirmado.

4.3 Critérios e forma de avaliação

- Correção funcional: respostas HTTP, persistência, controle de acesso e recálculo das recomendações.
- Integridade técnica: execução da suíte backend, typecheck do TypeScript e build de produção.

- Aderência de interface: inspeção qualitativa do fluxo, dos filtros, dos estados vazios e do feedback de ações.
- Qualidade da recomendação: fórmula e efeitos esperados dos sinais são testados; precisão em preferências reais não foi medida.
- Operação: configurações de ambiente e deploy estão descritas; não há evidência de deploy ou teste de carga em produção.

5. Persona e levantamento de necessidades

5.1 Proto-persona

Dimensão	Descrição
Persona	Marina, professora avaliadora (proto-persona hipotética).
Contexto	Recebe um link para avaliar um projeto acadêmico em computador ou celular.
Objetivo	Entender rapidamente a proposta e experimentar busca, filtros e recomendações.
Dores	Obrigatoriedade de criar conta; demonstração sem dados; recomendações sem explicação; telas confusas.
Necessidades	Acesso direto, catálogo navegável, preferência editável, feedback claro e transparência sobre limites.
Proposta de valor	Experimentar uma curadoria explicável sem cadastro, com ações simples e controle sobre os sinais.

A personagem não resulta de entrevista; é uma hipótese de design derivada do cenário informado para a avaliação docente. Antes de tratar necessidades como fatos, recomenda-se entrevistar docentes e estudantes, validar vocabulário, acessibilidade, tempo de conclusão e dificuldades reais.

5.2 Necessidades priorizadas

- N1 — iniciar a avaliação sem barreira de autenticação.
- N2 — compreender o que está sendo recomendado e por qual motivo.
- N3 — localizar itens por busca textual e filtros categóricos.

- N4 — expressar preferências e corrigir recomendações inadequadas.
- N5 — salvar favoritos e avaliações entre acessos ao mesmo perfil.
- N6 — saber que o modo de demonstração compartilha dados e não deve receber informação pessoal.
- N7 — manter a área administrativa protegida mesmo quando a aplicação é pública.

6. Especificação de requisitos

6.1 Requisitos funcionais

ID	Requisito	Critério de aceite / evidência
RF01	Acessar demonstração sem login quando DEMO_ACCESS=true.	auth/config informa demo_access=true; /auth/me retorna perfil demo.
RF02	Consultar e filtrar catálogo.	Listagem suporta consulta e filtros; frontend filtra tipo, gênero, idioma e texto.
RF03	Gerar recomendações ordenadas e explicáveis.	Endpoint retorna itens, estratégia, fórmula e motivo textual.
RF04	Editar e persistir preferências.	PUT valida estrutura e GET recupera tipos, gêneros, idiomas e diversidade.
RF05	Adicionar/remover favoritos e avaliar conteúdos.	Estado persiste; nota limitada a 1–5 e alimenta sinais.
RF06	Registrar visualizações e ocultar item.	Interações persistem; item oculto deixa de ser recomendado.
RF07	Limpar histórico de interações.	DELETE remove eventos de navegação sem excluir notas/favoritos.
RF08	Restringir métricas e cadastro administrativo.	Usuário sem papel autorizado recebe 403.
RF09	Exibir métricas administrativas agregadas.	Admin recebe totais, séries diárias, conteúdo ativo e atividade recente.
RF10	Permitir OAuth opcional.	Fluxo valida state/nonce e cria cookie HTTP-only após troca server-side.

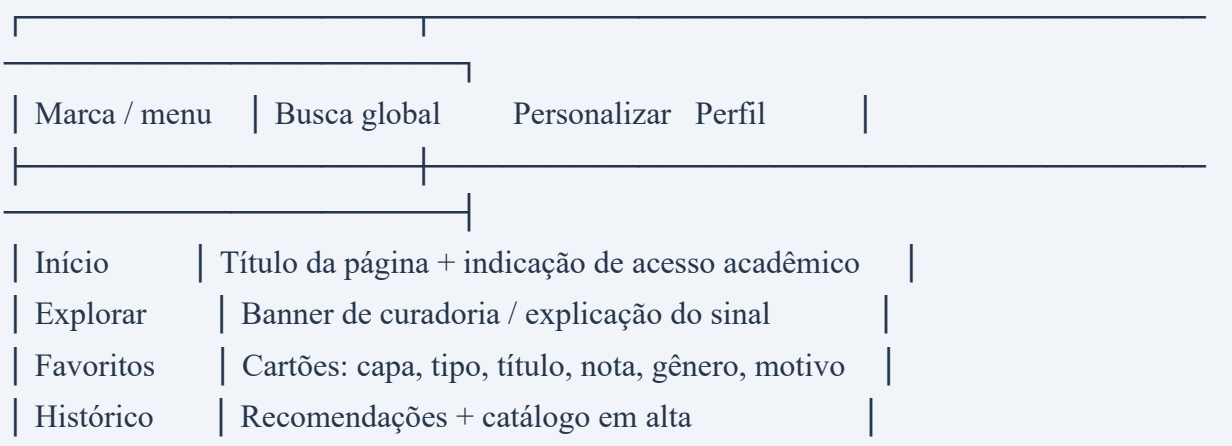
6.2 Requisitos não funcionais

ID	Atributo	Requisito / situação
RNF01	Usabilidade	Interface responsiva e linguagem em português; avaliação com usuários pendente.
RNF02	Segurança	Privilégio administrativo separado, cookie HTTP-only, validação JWT e checagem de origem em mutações.
RNF03	Privacidade	Perfil compartilhado no modo demo; não inserir dados pessoais; comportamento deve ser avisado ao avaliador.
RNF04	Manutenibilidade	Frontend tipado, API com modelos Pydantic e testes; módulo backend concentrado é risco de evolução.
RNF05	Portabilidade	Scripts de desenvolvimento e container; funcionamento depende das variáveis de ambiente e do runtime.
RNF06	Desempenho	Não há SLO nem teste de carga; catálogo pequeno não representa escala de produção.
RNF07	Disponibilidade	Deploy Vercel/Railway configurado em arquivos, mas implantação pública não foi verificada.
RNF08	Explicabilidade	Cada recomendação inclui texto de justificativa; não é explicação causal completa do modelo.

7. Prototipação

7.1 Wireframe conceitual

O protótipo está materializado na aplicação React; o esquema abaixo sintetiza a hierarquia funcional e não representa captura literal de tela:



```
| Preferências  | Filtros de tipo/gênero/idioma e diversidade  |
| Como recomenda | Detalhe: avaliar, favoritar, ocultar  |
└──────────────────┘──────────────────────────────────────────┘
```

7.2 Fluxo de navegação

```
Acesso direto → carregar auth/config → resolver perfil visitante → carregar catálogo,
preferências, favoritos e recomendações → explorar/buscar → abrir detalhe →
favoritar / avaliar / ocultar → persistir interação → recalcular recomendações.
```

O estado de autenticação ainda é consultado para manter compatibilidade com contas opcionais. Quando DEMO_ACCESS está ativo, falha por ausência ou expiração de sessão não deve bloquear o visitante; rotas administrativas continuam sujeitas à autorização no backend.

7.3 Decisões de usabilidade

- Busca global e filtros por chips tornam o catálogo localizável por diferentes estratégias.
- Cartões e modal de detalhes mantêm a lista escaneável e oferecem ações contextuais.
- Mensagens toast dão retorno após ações; telas vazias sugerem como prosseguir.
- Explicações em texto e seção “Como recomendamos” favorecem compreensão em vez de exibir apenas pontuação.
- A indicação de modo acadêmico deixa explícito que não há conta individual.
- A inspeção deve ser complementada com teclado, leitor de tela, contraste e teste responsivo em dispositivos reais.

8. Arquitetura e tecnologias

8.1 Visão lógica

```
Navegador (React + TypeScript)
    | HTTPS / JSON (mesma origem via proxy em deploy Vercel)
    ▼
```

```
API FastAPI ─ validação Pydantic ─ autenticação/autorização
        │
        ├── serviço de catálogo / preferências / feedback / métricas
        ├── ranking híbrido determinístico + registro de eventos
        ▼
SQLite (conteúdo, perfis, preferências, notas, favoritos e interações)
```

A camada frontend concentra rotas visuais e estado de tela em App.tsx, chamando a API por um serviço tipado. O backend organiza modelos, endpoints, autenticação, SQL e recomendação em backend/app/main.py; SQLite é acessado com sqlite3 e consultas parametrizadas. O framework oferece documentação OpenAPI. A arquitetura é adequada ao MVP e simples de executar, mas não apresenta separação física completa entre controller, serviço e repositório.

8.2 Modelo de dados

Entidade	Finalidade
contents	Catálogo: título, tipo, gêneros, idioma, popularidade e sinalizadores demonstrativos.
profiles	Identidade lógica, nome, email, papel e configuração de histórico.
preferences	Tipos, gêneros, idiomas, itens bloqueados e perfil de diversidade.
ratings / favorites	Notas de 1 a 5 e itens salvos por usuário.
interactions	Eventos de visualização, ocultação, avaliação e mudança de favorito.
recommendation_events	Contagem e estratégia associadas à geração de recomendações.

8.3 Algoritmo implementado

Para cada candidato elegível, o ranking calcula afinidade explícita E a partir da sobreposição com preferências de gênero, tipo e idioma; afinidade aprendida H a partir de avaliações, favoritos e visualizações do próprio perfil; qualidade média Q a partir das notas agregadas; e popularidade P. A expressão central é:

```
score = 0,44E + 0,34H + 0,10Q + 0,12P − penalidade_de_repetição + bônus_de_exploração
```

A implementação limita componentes, normaliza e restringe o score à faixa de 0 a 1, exclui conteúdos ocultos, aplica uma penalidade a itens já avaliados e tenta alternar tipos quando a diversidade não está em modo foco. A lógica produz

justificativas textuais. Como o perfil de visitante é compartilhado, sinais de diferentes avaliadores podem se misturar; esse comportamento é parte das limitações explícitas.

8.4 Implantação e configuração

Scripts npm/pnpm iniciam Vite e Uvicorn localmente. Dockerfile e Compose permitem execução em container; vercel.json e api/[...path].js encaminham API para Railway. DATABASE_URL, DEMO_ACCESS, CORS_ORIGINS, FRONTEND_URL e variáveis AUTH_* configuram execução. Essa preparação não comprova publicação: não há evidência de serviço externo provisionado, segredo de produção configurado ou deploy público validado.

9. Padrões de projeto

9.1 SOLID: aderência parcial e oportunidades

Princípio	Leitura no projeto
S — responsabilidade única	Há separação entre frontend e backend, mas main.py reúne rotas, autenticação, SQL e ranking; responsabilidade concentrada.
O — aberto/fechado	Modelos e contratos tipados dão base à extensão; novo algoritmo exigiria refatorar a função atual, não apenas adicionar uma estratégia.
L — substituição de Liskov	Não há hierarquia de estratégias ou contratos polimórficos relevantes para avaliar substituição.
I — segregação de interfaces	Contratos HTTP são relativamente focados; não há interfaces internas segregadas para persistência ou recomendação.
D — inversão de dependência	API depende diretamente de funções e sqlite3; abstração de repositório ainda não existe.

Assim, SOLID é referência de análise e direção futura, não afirmação de que todos os princípios estão implementados. A próxima evolução razoável é extrair serviço de recomendação e repositório com fronteiras pequenas somente quando houver novos casos de uso ou banco substituto.

9.2 Strategy

A resposta da API identifica a estratégia como HybridStrategy, e a fórmula implementa um ranking híbrido. No código atual, contudo, há uma função de recomendação principal; PopularityStrategy e ContentBasedStrategy independentes não estão implementadas como objetos intercambiáveis. Portanto, trata-se de uma estratégia funcional do produto e de uma aplicação inicial/incompleta do padrão GoF Strategy. Uma evolução pode definir contrato comum (por exemplo, recommend(profile, catalog)) e selecionar implementações por configuração, sem fazê-lo prematuramente.

9.3 Observer

A tabela recommendation_events registra que recomendações foram geradas, e interações ficam persistidas para consulta e métricas. Isso introduz observabilidade por eventos, mas não é uma implementação formal do Observer com sujeitos, assinantes e notificações desacopladas. A descrição correta é “registro persistido inspirado em eventos”; a separação entre evento de domínio e tabela analítica pode ser considerada quando existirem múltiplos consumidores.

10. Análise de riscos

Risco / ameaça	Prob.	Impacto	Mitigação atual / proposta	Residual
Dados de interação compartilhados no modo acadêmico	Alta	Médio	Avisar que o perfil é comum; não solicitar dado pessoal; oferecer reset de demonstração.	Médio
Configuração demo incorreta no ambiente	Média	Alto	Verificar auth/config após deploy e manter teste de regressão de acesso visitante.	Baixo/médio
Credencial OAuth/JWT exposta ou configurada incorretamente	Baixa/média	Alto	Segredos apenas no backend, HTTPS, rotação e escopo mínimo; revisão de configuração operacional.	Médio
Acesso	Baixa	Alto	Autorização no backend baseada em	Baixo/médio

administrativo indevido			lista ADMIN_OPEN_IDS; perfil demo sem papel admin; teste 403.	
SQLite limitado sob concorrência / perda de volume	Média	Médio/alto	Backup e volume persistente; monitorar bloqueios; migrar para PostgreSQL antes de escala.	Médio
Ranking reforça popularidade ou preferências estreitas	Média	Médio	Peso de popularidade limitado; modo explorador; medir diversidade e permitir ocultar item.	Médio
Dados fictícios e poucas interações distorcem avaliação	Alta	Médio	Rotular conteúdo demo; obter dataset consentido/licenciado e aumentar avaliação controlada.	Médio
Imagens externas indisponíveis ou sem licença adequada	Média	Médio	Revisar direitos, usar ativos próprios/licenciados ou placeholders locais e tratar falhas.	Baixo/médio
Ausência de teste de carga, acessibilidade e produção	Alta	Alto	Planejar testes antes de uso real; não declarar SLA ou adequação comercial.	Alto até validação

A análise é qualitativa: probabilidade e impacto são estimativas de projeto, não resultados de auditoria formal. A demonstração pública deve restringir informações e tráfego; para operação real, revisar ameaças, controles, retenção e obrigações aplicáveis da LGPD [9].

11. Viabilidade técnica e econômica

11.1 Viabilidade técnica

A solução é viável como MVP: dependências e ferramentas são amplamente disponíveis, SQLite evita serviço de banco dedicado no desenvolvimento, e execução local está automatizada. FastAPI, React, TypeScript e SQLite possuem documentação oficial e comunidades consolidadas [10–13]. O deploy em

Vercel/Railway é tecnicamente preparado por arquivos do projeto, porém a viabilidade em produção depende de teste de integração, persistência, variáveis secretas, domínio, monitoramento e custo atual dos provedores.

11.2 Viabilidade econômica

As ferramentas centrais são open source; não há custo de licença obrigatório no uso local. O principal custo acadêmico é mão de obra e infraestrutura de hospedagem. Valores monetários não são estimados porque planos, franquias gratuitas e região de nuvem mudam e não foram cotados nesta avaliação. A aplicação pública exige prever domínio, tráfego, armazenamento/backup e manutenção.

11.3 Cronograma indicativo para uma evolução de 5 semanas

Semana	Atividade	Entrega
1	Entrevistas rápidas, delimitação de requisitos e dados	Requisitos revisados e riscos priorizados.
2	Refino de navegação, wireframes e acessibilidade	Protótipo e fluxo testável.
3	Refatoração modular e melhoria de persistência	Serviços/repositórios e migrações avaliadas.
4	Dataset e avaliação offline/on-line de recomendação	Métricas, baselines e relatório experimental.
5	Deploy controlado, testes de segurança e documentação	Versão candidata, evidências e plano operacional.

Estimativa de esforço: 5 semanas com uma pessoa desenvolvedora em dedicação parcial; dependências externas e aprovações podem alterar o prazo. Trata-se de planejamento, não histórico de execução.

12. Trade-offs e critérios de avaliação

Trade-off	Escolha atual	Benefício	Custo / critério de revisão
Simplicidade × escala	SQLite e módulo backend único.	Setup fácil e baixa complexidade.	Contenção, migração e testabilidade; rever com

			concorrência/dados reais.
Transparência × sofisticação	Regras ponderadas e razões textuais.	Compreensíveis e reproduzíveis.	Menor capacidade de capturar contexto; comparar com baseline antes de sofisticar.
Personalização × privacidade	Perfil persistido e demo compartilhado.	Acesso sem conta e demonstração rápida.	Interações misturam avaliadores; separar perfis em uso real.
Afinidade × descoberta	Peso alto em preferências, mais ajuste explorador.	Usuário mantém controle e pode descobrir novos itens.	Pode haver bolha ou irrelevância; avaliar diversidade e utilidade.
Velocidade × validação	MVP funcional e dados fictícios.	Ciclo rápido de desenvolvimento.	Não demonstra eficácia externa nem disponibilidade de produção.

12.1 Critérios e métricas propostas

Dimensão	Métrica	Como medir / limite atual
Relevância	Precision@K / NDCG@K	Requer julgamento de relevância ou conjunto de teste; não calculado.
Cobertura	Itens recomendados distintos / catálogo elegível	Medir em população de perfis e períodos definidos; não calculado.
Diversidade	Intra-list diversity e distribuição de tipos/gêneros	Verificar variedade sem sacrificar relevância; não calculado.
Novidade	Popularidade média dos itens recomendados	Usar como proxy, não como benefício individual; não calculado.
Explicabilidade	Compreensão/adequação da justificativa	Questionário e entrevista; atualmente há texto de razão, sem estudo.
Desempenho	Latência p50/p95 sob carga definida	Executar teste de carga em ambiente equivalente à publicação; não medido.
Qualidade técnica	Testes, typecheck e build	Testes 10/10, TypeScript e build passaram nesta verificação.

Para comparação científica, deve-se definir uma baseline — por exemplo, ordenação por popularidade —, separar dados de treino e avaliação quando aplicável, registrar protocolo e evitar interpretar cliques como relevância sem validação. O catálogo atual de demonstração é insuficiente para alegar melhoria estatística ou precisão de produção.

13. Testes e resultados

13.1 Evidências executadas

Verificação	Resultado em 08/10/2026	Interpretação
Pytest: backend/tests	10 passaram; 0 falharam; cerca de 0,85 s.	Cobertura de fluxos selecionados, não cobertura total do código.
TypeScript: tsc --noEmit	Concluiu sem erros.	Não substitui teste funcional no navegador.
Vite: build de produção	Build concluído; 1.900 módulos transformados; JS 262,91 kB / gzip 81,79 kB.	Confirma compilação do frontend, não deploy externo.
Aviso observado	DeprecationWarning do alias anyio.abc.BlockingPortal via Starlette TestClient.	Aviso de dependência; não causou falha nos testes.

13.2 Escopo dos testes automatizados

Os 10 casos cobrem: health e catálogo; acesso anônimo acadêmico; sessão expirada e fallback visitante; rotas pessoais quando a demonstração é desativada; provisionamento de sessão válida; state/nonce e callback OAuth; explicabilidade do ranking; mudanças de sinais, nota negativa e ocultação; persistência de preferências e favoritos; e métricas administrativas com rótulo demonstrativo.

Os testes de integração usam TestClient e SQLite e comprovam contratos específicos no ambiente local. Eles não demonstram proteção contra ataques além dos casos simulados, não testam carga, disponibilidade de Vercel/Railway, acessibilidade formal ou satisfação de avaliadores. A autenticação real depende de credenciais e configuração externa que não são validadas pela suíte isolada.

13.3 Limitações dos resultados

- Dez testes não devem ser apresentados como cobertura de 100%; cobertura percentual não foi coletada.
- Nenhuma avaliação humana foi conduzida; a persona permanece hipotética.
- Não há conjunto de ground truth, Precision@K, NDCG, taxa de descoberta ou comparação estatística.
- O modo demo compartilha perfil e sinais, o que limita interpretações individuais.
- A compilação local não confirma deploy, secrets, domínio, banco persistente ou health check remoto.

14. Conclusão

O SmartRecomenda materializa um fluxo de descoberta que conecta preferências, sinais de interação, recomendação híbrida, explicações e persistência em uma aplicação executável. A interface oferece busca, filtros, favoritos, avaliações, preferências, histórico, perfil e área administrativa restrita. O acesso acadêmico sem login responde ao cenário de avaliação, com a ressalva explícita de que a identidade demonstrativa e suas interações são compartilhadas.

Na evidência local mais recente, a suíte do backend (10 casos), o typecheck e o build de produção passaram. Isso atende a uma verificação de integração e compilação para o escopo selecionado. Não permite concluir que as recomendações sejam precisas para usuários reais, que a interface seja usável por todos, ou que a versão hospedada esteja operacional.

As próximas melhorias prioritárias são:

1. validar tarefas e linguagem com professores e estudantes, incluindo acessibilidade por teclado e leitor de tela;
2. separar perfis de demonstração ou fornecer reset controlado e aviso de dados compartilhados;

3. aumentar e qualificar o catálogo, com origem e licença de dados e imagens documentadas;
4. extrair recomendador, serviços e persistência em módulos testáveis, implementando Strategy real apenas se novas estratégias forem necessárias;
5. definir dataset de relevância, baseline e métricas de precisão, cobertura, novidade e diversidade;
6. executar revisão de segurança, teste de carga, backups e validação de deploy em ambiente de homologação antes de produção.

Conclui-se que o projeto é tecnicamente viável como MVP acadêmico, oferece uma base demonstrativa consistente e deixa oportunidades de evolução bem identificadas. A principal contribuição está na integração de uma experiência acessível de avaliação com personalização explicável e controles de interação; a principal limitação é a ausência de validação empírica com usuários e dados representativos.

15. Referências

[1] RICCI, F.; ROKACH, L.; SHAPIRA, B. (ed.). Recommender Systems Handbook. 3. ed. Cham: Springer, 2022. DOI: https://doi.org/10.1007/978-1-0716-2197-4.

[2] BURKE, R. Hybrid recommender systems: survey and experiments. User Modeling and User-Adapted Interaction, v. 12, p. 331–370, 2002. DOI: https://doi.org/10.1023/A:1021240730564.

[3] BROWN, T. Design Thinking. Harvard Business Review, jun. 2008. Disponível em: https://hbr.org/2008/06/design-thinking. Acesso em: 8 out. 2026.

[4] ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering. Geneva: ISO, 2018.

[5] ISO/IEC. ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. Geneva: ISO, 2023.

[6] NIELSEN, J. 10 Usability Heuristics for User Interface Design. Nielsen Norman Group. Disponível em: https://www.nngroup.com/articles/ten-usability-heuristics/. Acesso em: 8 out. 2026.

[7] GAMMA, E. et al. Design Patterns: Elements of Reusable Object-Oriented Software. Boston: Addison-Wesley, 1994.

[8] MARTIN, R. C. Design Principles and Design Patterns. Object Mentor, 2000.

[9] BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

[10] FASTAPI. FastAPI documentation. Disponível em: https://fastapi.tiangolo.com/. Acesso em: 8 out. 2026.

[11] META OPEN SOURCE. React documentation. Disponível em: https://react.dev/. Acesso em: 8 out. 2026.

[12] SQLITE. SQLite documentation. Disponível em: https://sqlite.org/docs.html. Acesso em: 8 out. 2026.

[13] SMARTRECOMENDA. Código-fonte e documentação do projeto: README.md, backend/app/main.py, backend/tests/test_api.py e frontend/src/App.tsx. Versão local consultada em 8 out. 2026.